

Q2Static/Dynamic Library A navigation app needs a geometry library for computing magnitude and phase angle of complex numbers. Create both a static and dynamic version of the library and demonstrate that the program works with both. Write geometry.c with magnitude() (using $\sqrt{a^2+b^2}$) and phase() (using atan2()) Build static version (ar rcs) and dynamic version (-shared -fPIC) Show compilation command and output for both versions using same main.c

geometry.h

```
float magnitude(float a, float b);  
float phase(float a, float b);
```

geometry.c

```
#include<stdio.h>  
#include<math.h>  
#include "geometry.h"  
  
float magnitude(float a, float b) {  
    return sqrt(a*a + b*b);  
}  
  
float phase(float a, float b) {  
    return atan2(b, a);  
}
```

Main.c

```
#include <stdio.h>  
#include "geometry.h"  
  
int main()  
{  
    float real, imag;  
  
    printf("Enter Real Part: ");  
    scanf("%f", &real);  
  
    printf("Enter Imaginary Part: ");  
    scanf("%f", &imag);  
  
    printf("Magnitude = %f\n",  
        magnitude(real, imag));  
}
```

```

    printf("Phase = %f\n",
           phase(real, imag));

    return 0;
}

```

Link

// Static Link

1. gcc -c geometry.c -o geometry.o
2. ar rcs libgeometry.a geometry.o
3. gcc main.c -L. -lgeometry -lm -o staticprog
4. ./staticprog

// Dynamic Link

1. gcc -c -fPIC geometry.c -o geometry.o
2. gcc -shared -o libgeometry.so geometry.o -lm
3. export LD_LIBRARY_PATH=\$PWD:\$LD_LIBRARY_PATH
4. gcc main.c -L. -lgeometry -lm -o dynamicprog
5. ./dynamicprog

Q3System Calls A chat application needs two processes to communicate via a pipe. The parent sends a message "Hello from parent" and the child reads it and prints it in uppercase (converted manually character by character). Create a pipe using pipe(), then fork with fork() Parent closes read-end and writes; child closes write-end and reads Child converts each character using toupper() and prints the result

```

#include<stdio.h>
#include<unistd.h>
#include<ctype.h>
#include<string.h>

int main() {
    int fd[2];
    pid_t pid;
    char message[] = "Hello From Parent!";
    char buffer[100];
    pipe(fd);
    pid = fork();

```

```

    if(pid > 0) {
        close(fd[0]);
        write(fd[1], message, strlen(message)+1);
        close(fd[1]);
    } else if(pid == 0) {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        close(fd[0]);
        for(int i = 0; buffer[i]!='\0'; i++) {
            buffer[i] = toupper(buffer[i]);
        }
        printf("Message from parent in uppercase:\n");
        printf("%s\n",buffer);
    }
    return 0;
}

// gcc pipe.c -o pipeprog
// ./pipeprog

```

Q4 Multithreading A news aggregator fetches headlines from 3 different sources simultaneously. Simulate this by creating 3 threads, each printing 5 "headlines" (numbered lines) with a small delay, and verify interleaved execution. Create 3 threads, each receives a source name (e.g. "BBC", "CNN", "Reuters") as argument. Each thread prints its name + headline number 5 times with `usleep(100000)` between prints. Observe interleaved output; then add a mutex and observe serialized output — explain the difference.

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock;

// Function without mutex
void* headlines_without_mutex(void* arg)
{
    char* source = (char*)arg;

    for(int i = 1; i <= 5; i++)
    {
        printf("%s Headline %d\n", source, i);
    }
}

```

```

        usleep(100000);
    }

    return NULL;
}

// Function with mutex
void* headlines_with_mutex(void* arg)
{
    char* source = (char*)arg;

    pthread_mutex_lock(&lock);

    for(int i = 1; i <= 5; i++)
    {
        printf("%s Headline %d\n", source, i);

        usleep(100000);
    }

    pthread_mutex_unlock(&lock);

    return NULL;
}

int main()
{
    pthread_t t1, t2, t3;

    // WITHOUT MUTEX
    printf("----- Interleaved Output Without Mutex -----\\n\\n");

    pthread_create(&t1, NULL,
                  headlines_without_mutex, "BBC");

    pthread_create(&t2, NULL,
                  headlines_without_mutex, "CNN");

    pthread_create(&t3, NULL,
                  headlines_without_mutex, "Reuters");

    pthread_join(t1, NULL);

```

```

pthread_join(t2, NULL);
pthread_join(t3, NULL);

// WITH MUTEX
printf("\n----- Serialized Output With Mutex ----- \n\n");

pthread_mutex_init(&lock, NULL);

pthread_create(&t1, NULL,
               headlines_with_mutex, "BBC");

pthread_create(&t2, NULL,
               headlines_with_mutex, "CNN");

pthread_create(&t3, NULL,
               headlines_with_mutex, "Reuters");

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);

pthread_mutex_destroy(&lock);

return 0;
}

// gcc thread.c -o threadprog -lpthread
// ./threadprog

```

Q5CPU Scheduling A video rendering farm uses Round Robin scheduling with quantum = 3ms. Given 4 render jobs, simulate the scheduler and determine which job finishes first and the average turnaround time. Input: J1(bt=10), J2(bt=4), J3(bt=6), J4(bt=3) — all arrive at t=0 Simulate Round Robin with time quantum=3: use a queue and decrement burst time each round Print completion time for each job and calculate average turnaround time

```

#include <stdio.h>

int main()
{
    int bt[4] = {10, 4, 6, 3};

```

```

int rem_bt[4];
int ct[4];
int tq = 3;
int time = 0;
int completed = 0;
float avg_tat = 0;
for (int i = 0; i < 4; i++)
{
    rem_bt[i] = bt[i];
}
printf("Execution Order:\n");
while (completed < 4)
{
    for (int i = 0; i < 4; i++)
    {
        if (rem_bt[i] > 0)
        {
            if (rem_bt[i] > tq)
            {
                printf("J%d executed from %d to %d\n", i + 1, time,
time + tq);

                time = time + tq;
                rem_bt[i] = rem_bt[i] - tq;
            }
            else
            {
                printf("J%d executed from %d to %d\n", i + 1, time,
time + rem_bt[i]);

                time = time + rem_bt[i];
                ct[i] = time;
                rem_bt[i] = 0;
                completed++;
            }
        }
    }
}
printf("\nCompletion Time\n");
for (int i = 0; i < 4; i++)
{
    printf("J%d = %d\n", i + 1, ct[i]);
    avg_tat = avg_tat + ct[i];
}
avg_tat = avg_tat / 4;

```

```

printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
return 0;
}

```

Q6Deadlock Handling A cloud service has 4 virtual machines sharing 3 types of resources (CPU tokens, memory blocks, network slots). Use the Banker's Algorithm to determine if a new resource request from VM2 can be safely granted. Set up Allocation, Max, Available for 4 processes and 3 resource types of your choice VM2 requests (1,0,2) — first check if $\text{request} \leq \text{Need}$ and $\leq \text{Available}$ Simulate granting request and run safety algorithm; print "Request granted" or "Request denied"

```

#include <stdio.h>
int main()
{
    int allocation[4][3] = {
        {1, 1, 0},
        {1, 0, 2},
        {1, 3, 1},
        {0, 2, 1}};
    int max[4][3] = {
        {3, 2, 2},
        {2, 1, 3},
        {1, 4, 2},
        {1, 2, 2}};
    int available[3] = {2, 1, 1};
    int need[4][3];
    int request[3] = {1, 0, 1};
    int finish[4] = {0};
    int safe[4];
    int work[3];
    int count = 0;
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }
    printf("\nNeed Matrix:\n");
    for (int i = 0; i < 4; i++)

```

```

{
    for (int j = 0; j < 3; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}
for (int j = 0; j < 3; j++)
{
    if (request[j] > need[1][j])
    {
        printf("Request Denied! ");
        return 0;
    }
}
for (int j = 0; j < 3; j++)
{
    if (request[j] > available[j])
    {
        printf("Request Denied! ");
        return 0;
    }
}
for (int j = 0; j < 3; j++)
{
    available[j] = available[j] - request[j];
    allocation[1][j] = allocation[1][j] + request[j];
    need[1][j] = need[1][j] - request[j];
}
for (int j = 0; j < 3; j++)
{
    work[j] = available[j];
}
while (count < 4)
{
    int found = 0;
    for (int i = 0; i < 4; i++)
    {
        if (finish[i] == 0)
        {
            int possible = 1;
            for (int j = 0; j < 3; j++)
            {

```

```
        if (need[i][j] > work[j])
        {
            possible = 0;
            break;
        }
    }
    if (possible)
    {
        for (int j = 0; j < 3; j++)
        {
            work[j] = work[j] + allocation[i][j];
        }
        safe[count] = i;
        finish[i] = 1;
        found = 1;
        count++;
    }
}

if (found == 0)
{
    break;
}

if (count == 4)
{
    printf("\nRequest Granted!\n");
    printf("\nSafe Sequence: ");
    for (int i = 0; i < 4; i++)
    {
        printf("VM%d ", safe[i]);
    }
    printf("\n");
}
else
{
    printf("\nRequest Denied!\n");
}

return 0;
}
```

Q7Shared Memory A multiplayer game server uses shared memory to track a player's score. One process (game engine) updates the score and another process (leaderboard UI) displays it. Use POSIX shared memory for this. Use `shm_open()`, `ftruncate()`, and `mmap()` to create a shared score variable. Writer process sets score to 100, then 200, then 300 with a 1-second delay between each. Reader process reads and prints score 3 times; both clean up using `munmap()` and `shm_unlink()`.

share1.c

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main()
{
    int shm_fd;

    int *score;

    // Create shared memory
    shm_fd = shm_open("/score", O_CREAT | O_RDWR, 0666);

    // Set size
    ftruncate(shm_fd, sizeof(int));

    // Map memory
    score = mmap(0,
                 sizeof(int),
                 PROT_READ | PROT_WRITE,
                 MAP_SHARED,
                 shm_fd,
                 0);

    // Write score values
    *score = 100;
    printf("Writer wrote: %d\n", *score);

    sleep(1);

    *score = 200;
    printf("Writer wrote: %d\n", *score);
```

```

    sleep(1);

    *score = 300;
    printf("Writer wrote: %d\n", *score);

    sleep(1);

    // Remove mapping
    munmap(score, sizeof(int));

    return 0;
}

```

share2.c

```

#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main()
{
    int shm_fd;

    int *score;

    // Open shared memory
    shm_fd = shm_open("/score", O_RDONLY, 0666);

    // Map memory
    score = mmap(0,
                 sizeof(int),
                 PROT_READ,
                 MAP_SHARED,
                 shm_fd,
                 0);

    // Read score values
    for(int i = 0; i < 3; i++)
    {

```

```

        printf("Reader read: %d\n", *score);

        sleep(1);
    }

    // Remove mapping
    munmap(score, sizeof(int));

    // Delete shared memory
    shm_unlink("/score");

    return 0;
}

```

Q8IPC Classical Problems A newspaper office has journalists (writers) and editors (readers). Multiple editors can review simultaneously, but a journalist needs exclusive access to write. Implement the Readers-Writers problem using semaphores. Use semaphores `mutex` and `write_lock`; track `reader_count` as shared variable. Implement 2 reader threads and 1 writer thread; each runs 3 iterations. Print which process is reading/writing at each step and verify no writer enters while readers are active.

```

#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

sem_t mutex;
sem_t write_lock;
int reader_count = 0;
void *reader(void *arg)
{
    int id = *(int *)arg;
    for (int i = 1; i <= 3; i++)
    {
        sem_wait(&mutex);
        reader_count++;
        if (reader_count == 1)
        {
            sem_wait(&write_lock);

```

```

    }
    sem_post(&mutex);
    printf("Reader %d is reading\n", id);
    sleep(1);
    sem_wait(&mutex);
    reader_count--;
    if (reader_count == 0)
    {
        sem_post(&write_lock);
    }
    sem_post(&mutex);
    sleep(1);
}
return NULL;
}

void *writer(void *arg)
{
    for (int i = 1; i <= 3; i++)
    {
        sem_wait(&write_lock);
        printf("Writer is Writing\n");
        sleep(1);
        sem_post(&write_lock);
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_t r1, r2, w1;
    int id1 = 1;
    int id2 = 2;
    sem_init(&mutex, 0, 1);
    sem_init(&write_lock, 0, 1);
    pthread_create(&r1, NULL, reader, &id1);
    pthread_create(&r2, NULL, reader, &id2);
    pthread_create(&w1, NULL, writer, NULL);
    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(w1, NULL);
    sem_destroy(&mutex);

```

```

sem_destroy(&write_lock);
return 0;
}

```

Q9Disk Scheduling A cloud backup service receives disk I/O requests at tracks: 55, 58, 39, 18, 90, 160, 150, 38, 184. Head starts at 100 moving toward lower tracks. Apply C-SCAN algorithm and find total head movement. Implement C-SCAN: head moves in one direction, jumps to opposite end, continues same direction. Print service order and mark where the head jumps. Calculate total head movement including the jump distance.

```

#include <stdio.h>
int main()
{
    int request[] = {55, 58, 39, 18, 90, 160, 150, 38, 184};
    int n = 9;
    int head = 100;
    int total = 0;
    int disk_size = 199;
    printf("Service Order:\n");
    int lower[] = {90, 58, 55, 39, 38, 18};
    int higher[] = {184, 160, 150};
    for (int i = 0; i < 6; i++)
    {
        printf("%d -> %d\n", head, lower[i]);
        total = total + (head - lower[i]);
        head = lower[i];
    }
    printf("%d -> 0\n", head);
    total = total + head;
    head = 0;
    printf("JUMP 0 -> %d\n", disk_size);
    total = total + disk_size;
    head = disk_size;
    for (int i = 0; i < 3; i++)
    {
        printf("%d -> %d\n", head, higher[i]);
        total = total + (head - higher[i]);
        head = higher[i];
    }
    printf("\nTotal Head Movement = %d", total);
}

```

```
    return 0;
}
```

Q10Memory Address Translation A web browser uses a two-level page table for memory isolation between tabs. Given a virtual address and a 2-level page table structure, compute the physical address step by step. Virtual address is 32-bit: bits [31:22] = L1 index, bits [21:12] = L2 index, bits [11:0] = offset Given virtual address 0x00403004: extract L1=1, L2=3, offset=4 manually, look up in provided tables L1[1] → L2 table base = 5000; L2[3] → frame = 7; compute and print physical address = $7 \times 4096 + 4$

```
#include <stdio.h>
int main()
{
    unsigned int virtual_address = 0x00403004;
    unsigned int l1_index;
    unsigned int l2_index;
    unsigned int offset;
    unsigned int l1_table[1024];
    unsigned int l2_table[1024];
    unsigned int frame;
    unsigned int physical_address;
    l1_index = (virtual_address >> 22) & 0x3FF;
    l2_index = (virtual_address >> 12) & 0x3FF;
    offset = (virtual_address) & 0xFFF;
    l1_table[1] = 5000;
    l2_table[3] = 7;
    printf("\nVirtual Address = 0x%X", virtual_address);
    printf("\nL1 Index = %u ", l1_index);
    printf("\nL2 Index = %u ", l2_index);
    printf("\nOffset = %u ", offset);
    printf("\nL1[%u] -> L2 base = %u ", l1_index, l1_table[l1_index]);
    frame = l2_table[l2_index];
    printf("\nL2[%u] -> frame = %u ", l2_index, frame);
    physical_address = (frame * 4096) + offset;
    printf("\nPhysical Address = %u", physical_address);
    return 0;
}
```